

Table 1: The evolution of software operations

Stage	Primary focus	Role of AI	Decision making
Traditional IT	Manual operations	None	Human
DevOps	Automation and CI/CD	Limited	Predefined rules
AIOps	Intelligent monitoring	Analytics and ML	AI-assisted
Agentic DevOps	Autonomous operations	Reasoning + Agents	Agent + Human
Autonomous platform	Self-managing systems	Multi-agent intelligence	Controlled autonomy

with different systems, and carry out actions aligned to a specific objective. That creates a more dynamic way of operating, where the whole system is not locked into one fixed sequence of instructions.

Here's an example of a traditional pipeline vs an agentic workflow:

Traditional CI/CD approach:

```
deploy:
  steps:
    - build
    - test
    - deploy

rollback:
  if: deployment_failed
  action: rollback
```

Agentic approach:

```
def agentic_devops(event):

    context = observe(
        logs=True,
        metrics=True,
        deployment_history=True
    )

    problem = analyze(context)

    plan = create_plan(
        objective="restore_application_
health",
        problem=problem
    )

    if requires_approval(plan):
        request_human_approval(plan)
```

```
result = execute(plan)
```

```
verify(result)
```

Here's another smaller code example:

```
@tool
def check_kubernetes_pods(namespace):
    """Check the health of pods in a
    Kubernetes namespace."""
    return kubectl(
        f"get pods -n {namespace}"
    )

agent = create_agent(
    tools=[check_kubernetes_pods],
    goal="Investigate unhealthy
application pods"
)
```

This demonstrates a key idea of Agentic DevOps: AI + reasoning + tools.

Automation and agentic intelligence

Automation and agentic intelligence seem similar but are not the same. Automation tends to follow instructions

that already exist — the path is pretty much written ahead of time. An agent, on the other hand, aims at some goal and then figures out what steps are needed to get there.

Think about a simple deployment rule: 'If the deployment fails, automatically roll back to the previous version'. That's meant for automation because the trigger and the reaction are known ahead of time, and there are no surprises.

Picture a case where the deployment technically 'works', yet application performance starts to slip right after. CPU usage climbs, database connections begin failing, and users get hit with high latency. There may not be any single predefined rule to cope with this scenario.

So you move to a different operational loop: observe, understand, plan, act, and verify. First, the system observes what's happening, then it builds an understanding of the situation, makes a plan, takes action, and finally checks whether the action actually delivered the desired outcome.

Table 2: Traditional automation vs agentic DevOps

Traditional automation	Agentic DevOps
Follows predefined rules	Works towards defined goals
Executes fixed workflows	Dynamically plans actions
Handles known scenarios	Can investigate unfamiliar scenarios
Limited contextual understanding	Context-aware reasoning
Requires predefined responses	Can select from multiple actions
Usually stops after execution	Verifies the outcome
Human handles exceptions	Agent can handle approved exceptions